



南京大學

本科畢業論文

院 系 計算機科學與技術系

專 業 計算機科學與技術

題 目 SMT 求解器在浮點數約束問題中
的性能評估與改進研究

年 級 2020 學 號 201220197

學生姓名 崔晨琦

指導教師 王豫 職 稱 助理研究員

提交日期 2024 年 5 月 13 日

南京大学本科生毕业论文（设计、作品）中文摘要

题目：SMT 求解器在浮点数约束问题中的性能评估与改进研究

院系：计算机科学与技术系

专业：计算机科学与技术

本科生姓名：崔晨琦

指导教师（姓名、职称）：王豫 助理研究员

摘要：

可满足性模理论 (Satisfiability Modulo Theories, SMT) 是在特定背景理论下判断一阶逻辑公式是否可满足的问题，在软件测试、程序验证、静态分析等许多领域都有应用，这些领域的很多问题都可以被转变为 SMT 擅长描述和求解的约束可满足性问题。浮点数背景理论是 SMT 的背景理论之一，是一种重要理论，实际问题中常常会出现数据用浮点数表示的情况，因此 SMT 求解器经常需要求解浮点数约束问题。但由于浮点数背景理论的复杂语义和精度缺陷等原因，SMT 求解器在面对一些复杂的、非线性的浮点数约束时通常会遇到困难、表现不佳。

为了了解和探索当前的 SMT 求解器对浮点数约束问题的性能表现和能力范围，本文选取了 4 个 SMT 求解器进行性能评估，分别是当前表现最好的主流 SMT 求解器之一 Z3 与针对浮点数约束问题的 JFS, XSat 和 goSAT。从这 4 个 SMT 求解器在选取的浮点数约束测试数据集上的实验结果来看，Z3 这类主流 SMT 求解器具有较好的稳定性，能应对更大范围的问题，而 JFS, XSat 和 goSAT 这类将浮点数约束问题转变为优化问题等其他问题的 SMT 求解器具有更少的求解时间和更多的求解数目。

本文根据 XSat 将求解浮点数约束问题转变为求目标函数最小值点问题的基本原理和 XSat 在实际实现中的可改进之处，提出了两种基于启发式策略的帮助求最小值点的改进方法，并设计了 4 组实验进行测试，证明了这两种方法及其组合能有效减少求解时间。

关键词：可满足性模理论 (SMT)；SMT 求解器；浮点数约束；启发式策略

南京大学本科生毕业论文（设计、作品）英文摘要

THESIS: Evaluation and Improvement of SMT Solvers in Solving Floating-Point Constraints

DEPARTMENT: Department of Computer Science and Technology

SPECIALIZATION: Computer Science and Technology

UNDERGRADUATE: Chenqi Cui

MENTOR: Assistant researcher Yu Wang

ABSTRACT:

Satisfiability Modulo Theories(SMT) are the problems of determining the satisfiability of first-order logic formulas with specific background theories, which can be applied in many fields, such as software testing, program verification and static analysis. Many problems in these fields can be transformed to constraint satisfiability problems which SMT is good at expressing and solving. Floating-point background theory is one of the background theories of SMT, and it is an important theory. In practical problems, data is often represented as floating-point numbers, so SMT solvers often need to solve floating-point constraints. But due to the complex semantics, precision deficiency and other problems of the floating-point background theory, SMT solvers often run into difficulties when solving complex, non-linear floating-point constraints.

In order to research and explore the performance and capability of current SMT solvers when solving floating-point constraints, this paper selected four SMT solvers for evaluation, namely one of the state-of-the-art SMT solvers, Z3, and three SMT solvers which are focus on floating-point constraints, JFS, XSat and goSAT. From the experimental results of these SMT solvers on the selected floating-point constraints benchmarks, it can be seen that the state-of-the-art SMT solvers, such as Z3, have better stability and capability, while SMT solvers which transform floating-point constraints to optimization problems and other problems, such as JFS, XSat and goSAT, have less solving time and more solving results.

In this paper, according to the idea of XSat transforming floating-point constraints to the problem of searching for the minimum point of objective functions and the de-

iciencies of XSat in implementation, two methods based on heuristic strategy to help find the minimum point are proposed. Four experiences are designed to test the two methods, proving that these methods and their combination can effectively reduce the solving time.

KEYWORDS: Satisfiability Modulo Theories (SMT); SMT solver; floating-point constraints; heuristic

目 录

第一章 绪论	1
1.1 研究背景及意义	1
1.2 相关工作	2
1.2.1 SMT 求解技术	2
1.2.2 SMT 求解器	3
1.2.3 SMT-LIB 和 SMT-COMP	4
1.3 本文研究内容	5
1.4 本文结构	5
第二章 背景知识	7
2.1 SMT 相关概念	7
2.1.1 SAT	7
2.1.2 SMT	8
2.2 浮点数背景理论	9
2.3 优化问题及其算法	11
2.4 启发式策略	11
第三章 SMT 求解器的评估	13
3.1 待评估的 SMT 求解器	13
3.1.1 Z3	13
3.1.2 JFS	14
3.1.3 XSat	15
3.1.4 goSAT	16
3.2 测试数据集	17
3.3 评估方法	17

第四章 SMT 求解器的改进研究	19
4.1 XSat 可改进的地方	19
4.2 改进方法与实现	19
4.2.1 目标函数的代码形式	19
4.2.2 比例过滤	20
4.2.3 迭代更新	23
4.2.4 两个方法的组合	24
第五章 实验分析	27
5.1 实验环境	27
5.2 评估分析	27
5.2.1 SMT 求解器配置	28
5.2.2 测试数据集分析	28
5.2.3 评估结果与分析	29
5.3 改进研究	32
5.3.1 实验设计	32
5.3.2 效果分析	35
第六章 总结与展望	39
6.1 总结	39
6.2 不足与展望	39
参考文献	41
致 谢	45

第一章 绪论

1.1 研究背景及意义

随着科技不断发展，人类社会进入信息化时代，越来越多的计算机软硬件走入人们的工作和生活中。在计算机技术不断进步的同时，软、硬件也变得越来越复杂，如何确保软、硬件的正确性和可靠性成为计算机领域的重要研究问题之一。在 20 世纪 80 到 90 年代，人们开始重视用形式化方法 (Formal Methods) 对软、硬件进行开发和验证，进而出现了包括以布尔可满足问题 (Boolean Satisfiability Problem, SAT) 和可满足性模理论 (Satisfiability Modulo Theories, SMT) 为代表的约束求解的形式化验证技术，并开发出了相关工程化工具^[1]。如今，SMT 求解技术和求解器已经被广泛运用于程序缺陷的检测与验证、静态分析、自动生成测试用例、RTL 验证、云计算和云存储等领域^[2]，具有很大研究意义。

SMT 是在 SAT 的基础上发展而来的。SAT 问题研究命题逻辑公式的可满足性，即给定一个布尔表达式，能否找到某种布尔变量的赋值方式使得表达式为真。SAT 在 1971 年被证明为非确定性多项式完全 (Non-deterministic Polynomial Complete, NPC) 问题，也是第一个被证明了的 NPC 问题，在硬件测试、电路验证等许多领域都有广泛应用，很多实际问题都可以转化为 SAT 问题^[2]。但随着 SAT 的深入研究和应用，人们发现只面向命题逻辑的 SAT 的表达能力较为有限，很多实际问题在转变为 SAT 问题进行求解时会丢失信息，这给人们带来了不便，也会让结果不准确，同时还会增大问题规模和复杂性^[3]，所以人们将 SAT 问题扩展为表达能力更强的 SMT 问题。SMT 在 SAT 的基础上结合了背景理论，将命题逻辑公式扩充为一阶逻辑公式，补充了量词和项等内容，公式中的命题变量可以解释为背景理论公式，因此 SMT 具有更强的表达能力和更高的抽象层次^[2,4]。SMT 的常用背景理论包括整数、实数、浮点数、线性算术、非线性算术、数组、位向量、字符串、未解释函数、差分逻辑等理论和它们的组合。

为了解决 SMT 问题，人们开始研究 SMT 求解技术，并研发了许多 SMT 求

解器。目前 SMT 求解技术已经对一些背景理论以及这些背景理论的部分组合有了优秀的判定方法，但仍存在局限性，例如对于带量词的或非线性算术领域的一阶逻辑公式尚未有完全判定方法^[5]。浮点数背景理论的 SMT 求解问题，即浮点数约束问题正是目前 SMT 求解技术会陷入困难的问题之一。浮点数背景理论是一种重要理论，实际问题中经常会遇到，在测试用例自动生成、程序合成等领域都有应用^[6]，但 SMT 求解器在面对复杂的、非线性的浮点数约束时，由于浮点数理论的复杂语义、精度缺陷等问题，常常会表现不佳^[6-7]，因此评估和提升 SMT 求解器对浮点数约束的求解性能是十分有意义的。

1.2 相关工作

SMT 求解技术的研究起源于 20 世纪 70 年代末 80 年代初，在 20 世纪 90 年代，人们开始研究大规模问题的 SMT 求解技术^[8]。如今 SMT 求解技术经历了蓬勃的发展，许多 SMT 求解器被研发出来，部分成熟的 SMT 求解器还被集成到了大型项目工具中，在学术界和工业界得到广泛应用。

1.2.1 SMT 求解技术

在 SMT 求解技术的发展过程中，主要出现了三类算法策略，分别是积极类算法（Eager 策略）^[9]和惰性算法（Lazy 策略）^[10]，以及 DPLL(T) 策略^[11]。

Eager 策略是在 SMT 求解技术发展早期提出的方法，它首先根据不同背景理论采用不同编码方式将一阶逻辑公式转变为等价的命题逻辑公式，即把 SMT 问题转变为等价的 SAT 问题，再用 SAT 求解器进行求解。这一策略的优点是不用为 SMT 多种多样的背景理论开发理论求解器，缺点是过于依赖编码方式和 SAT 求解器的正确性，而且在处理规模稍大的问题时生成的公式长度会指数级爆炸增长，因此难以应用到工业界中^[2]。

Lazy 策略则是先在不考虑背景理论的情况下将 SMT 公式当作 SAT 公式求解，若 SAT 公式有解再利用理论求解器结合背景理论考察解的可行性，否则可直接说明 SMT 公式无解。Lazy 策略虽然需要专门设计理论求解器，但相比于 Eager 策略具有更好的扩展性，能更加灵活地处理大规模问题，因此很长一段时间的 SMT 求解器都采用 Lazy 策略^[5]。

DPLL(T) 策略是基于 DPLL (Davis-Putnam-Logemann-Loveland) 算法框架的求解策略, 将原本用于解决 SAT 问题的 DPLL 算法框架拓展到 SMT 领域, 可用于建模实现 Lazy 策略的一些变体, 并同时具备 Eager 策略的高效性和 Lazy 策略的灵活性^[11]。DPLL 的基本思想是选择一个没有被赋值的命题变量进行赋值, 然后进行推导, 如果没有冲突、能推导为真则 SAT 公式可解, 否则进行回溯, 若不能回溯则说明 SAT 公式不可解。DPLL(T) 在 DPLL 的基础上加入了可替换的理论求解器部分, 在推导时检查是否满足背景理论, 并可以嵌入许多高效的启发式策略, 因而具有灵活的通用性和更高的效率, 目前主流的 SMT 求解器大多采用 DPLL(T) 策略^[4]。

1.2.2 SMT 求解器

SMT 求解器是 SMT 求解技术的具体实现^[2]。随着 SMT 求解技术的不断发展, 很多研究人员和科研机构积极开发和改进了许多 SMT 求解器。目前主流的 SMT 求解器包括 Z3^[3], CVC5^[12], MathSAT5^[13], Yices2^[14], Boolector^[15], Bitwuzla^[16]等, 它们基本都支持多种背景理论及其组合, 具有较好的求解能力, 有些已被集成在大型项目工具中, 在许多领域发挥良好作用。另外也有专门研究某一种背景理论的 SMT 求解器, 这些求解器在特定的背景理论下有时也会有很好的表现, 例如针对浮点数背景理论的 JFS^[17], XSat^[7]和 goSAT^[6], 它们将浮点数约束问题转变为优化问题 (optimization problem) 等其他问题, 再使用现成的工具求解, 从而避免了直接针对约束本身进行求解, 因此具有更高的求解速度^[18]。Z3, JFS, XSat, goSAT 是本文待评估的 SMT 求解器, 将在第三章详细介绍, 这里简单介绍当前表现较好的 CVC5, MathSAT5, Yices2 和 Bitwuzla。

CVC5 是 Barbosa 等人^[12]在 CVC3 和 CVC4 的基础上开发的新求解器, 采用了基于 DPLL 的改进算法 CDCL (Conflict-Driven Clause Learning Algorithm) 的 CDCL(T) 框架^[19]。CVC5 的 SMT Solver 模块由 4 部分组成, 分别是: 对输入进行预处理的 Preprocessor、对公式进行抽象重写的 Rewriter、对抽象后的公式进行求解的 Propositional Engine、检查解是否满足背景理论的 Theory Engine。CVC5 支持多种背景理论及其组合, 对无量词和量化公式都能很好地解决, 在近年的 SMT-COMP 的多个赛道都取得了优秀成绩, 目前已在 Boogie, SPARK 等大型项目中充当后端求解器^[12]。

MathSAT5 是 Cimatti 等人^[13]在 MathSAT4 的基础上改进而来的新求解器, 利用 DPLL(T) 框架和理论求解器进行求解, 支持线性算术、位向量、数组、未解释函数等大部分背景理论及其组合, 并能为不可满足的 SMT 公式提供具体的驳斥证明^[4]。MathSAT5 同样在 SMT-COMP 取得过优异成绩, 并已在工业界中得到应用, 例如在 Intel 的 RTL 形式化验证工具中充当后端工具^[13]。

Yices2 是斯坦福大学的 SRI International 研究院^[14]在 Yices 的基础上改进开发的 SMT 求解器, 基于 CDCL(T) 框架进行求解, 支持整数和实数的线性算术、位向量、数组、未解释函数、差分逻辑等多种背景理论及其组合, 并对求解组合背景理论的方法做出了改进。

Bitwuzla 是 Niemetz 等人^[16]为了解决 Boolector 在架构上存在的局限而从零开始重新开发的 SMT 求解器, 主要针对位向量、数组、浮点数、未解释函数等背景理论的无量词和量化公式。Bitwuzla 的主要组件包括 Node Manager 和 Solving Context, 前者为后者构建数据结点, 后者通过实现一个惰性 SMT 范式 Lemmas on Demand^[20]对 SMT 公式进行求解, 这也是 Bitwuzla 相比于 CVC5, MathSAT5 和 Yices2 等基于 DPLL(T) 和 CDCL(T) 框架的求解器的主要区别。

1.2.3 SMT-LIB 和 SMT-COMP

除了 SMT 求解器自身的进步, SMT-LIB (Satisfiability Modulo Theories Library) 标准^[21]和 SMT-COMP (International Satisfiability Modulo Theories Competition) 竞赛^[22]也为 SMT 求解器的发展起到了很大作用。SMT-LIB 标准是目前公认程度较高的 SMT 研究标准^[2], 它规定了 SMT 求解器输入输出的语言规范, 对 SMT 的不同背景理论进行分类, 例如 QF_BV (无量词位向量理论)、QF_AX (无量词数组理论)、QF_FP (无量词浮点数理论) 等, 并定义了数据的表示形式、对数据的各种操作等内容, 形成了可基于不同背景理论的格式统一的测试集, 使得对 SMT 求解器求解能力的评估和比较有了良好的标准。SMT-COMP 竞赛是可满足性理论及其应用国际学术年会每年举办的当前 SMT 领域公认度最高的比赛^[2], 利用来源于 SMT-LIB 的标准测试用例集分不同赛道对 SMT 求解器的性能进行测试和评分, 极大激发了研究者们对 SMT 求解器的热情。

1.3 本文研究内容

正如前文介绍，目前有许多 SMT 求解技术和求解器，并且当前的 SMT 求解器在求解浮点数约束问题时还存在着困难。为了研究当前的 SMT 求解器对浮点数约束问题的性能表现和能力范围，探索进一步提升 SMT 求解器对浮点数约束问题的求解效果的方法，本文做了如下研究内容：

1. 选取当前表现较好的主流 SMT 求解器 Z3 和针对浮点数约束问题的 JFS, XSat, goSAT, 详细了解它们的实现原理，并挑选浮点数背景理论的数据集对选取的 SMT 求解器进行测试，对它们在浮点数约束问题上的求解表现进行性能评估。
2. 针对评估的 SMT 求解器之一 XSat 的实现原理和其在实际实现中存在的可改进之处思考和实现了基于启发式策略的改进方法，并设计平行实验测试了改进效果。

1.4 本文结构

本文共 6 个章节，结构内容如下：

第一章为绪论，首先介绍了 SMT 的发展历程和研究意义，接着介绍了 SMT 的相关工作，包括 SMT 求解技术和 SMT 求解器的研究现状等，最后明确了本文的研究内容和结构。

第二章为背景知识，主要介绍了 SMT 的理论知识，重点介绍了 SMT 背景理论中的浮点数背景理论，还介绍了优化问题和启发式策略的相关概念与算法。

第三章为 SMT 求解器的评估，详细介绍了待评估的 4 个 SMT 求解器：Z3、JFS、XSat、goSAT 的基本原理和主要特点，并介绍了选取的测试数据集以及评估方法。

第四章为 SMT 求解器的改进研究，介绍了 XSat 在实际实现中的可改进之处，并提出了两个基于启发式策略的改进方法，描述了它们的具体实现方式。

第五章为实验分析，首先介绍了实验配置，接着介绍了 SMT 求解器的评估结果，并进行了分析，最后展示了第四章提出的两个改进方法及其组合的改进效果。

第六章为总结与展望，总结了本文做的研究工作，指出了目前存在的不足之处，并对未来的研究工作进行了展望。

第二章 背景知识

本文研究关于浮点数背景理论的 SMT 求解技术，并尝试利用启发式策略帮助提高将浮点数约束问题转变为优化问题的 SMT 求解器的求解效果，因此本章将依次介绍 SMT 的相关概念和浮点数背景理论，以及优化问题与启发式策略的相关知识和算法。

2.1 SMT 相关概念

SMT 问题是判定结合了背景理论的一阶逻辑公式是否可满足的问题，是对 SAT 问题的扩充，因此本节先从 SAT 的相关概念开始介绍。

2.1.1 SAT

SAT 的研究对象是命题逻辑公式 (propositional logical formula)，即由取值为真 (true) 或假 (false) 的布尔变量 (称为命题变元) 和与 ($\wedge$)、或 ($\vee$)、非 ($\neg$)、蕴含 ($\rightarrow$)、等价 ($\leftrightarrow$)、异或 ($\oplus$) 等布尔逻辑运算符组成的公式。命题逻辑公式具有以下定义和规则^[2]：

1. 单独的命题变元也是命题逻辑公式，此时也可称为原子公式；
2. 若 f 是命题逻辑公式，则 $\neg f$ 也是命题逻辑公式；
3. 若 f_1 和 f_2 是命题逻辑公式，则 $f_1 \perp f_2$ 也是命题逻辑公式，其中 $\perp$ 指 $\wedge$, $\vee$, $\neg$, $\rightarrow$, $\leftrightarrow$, $\oplus$ 等；
4. 命题变元 f 及其否定 $\neg f$ 统称为文字，若干文字的析取 ($\vee$) 称为一个子句，若干子句的合取 ($\wedge$) 称为一个合取范式 (Conjunctive Normal Form, CNF)；
5. 给命题逻辑公式中的所有命题变元赋值为真或者假的一种赋值方案称为一种指派，一种指派连同以此得到的命题逻辑公式的真值称为该公式的一种解释。

在以上关于命题逻辑公式的定义和规则的基础上，SAT 问题可以描述为：给定一个 CNF 形式的命题逻辑公式 f ，判断是否存在一种指派使得 f 为真，若存在则说明 f 是可满足的 (satisfiable, sat)，否则说明 f 是不可满足的 (unsatisfiable, unsat)。

2.1.2 SMT

SMT 的研究对象是一阶逻辑公式，它在命题逻辑公式的基础上加入了量词、谓词、函数、项等如下内容^[4]：

1. 量词 (quantifier) 包括全称量词 ($\forall$) 和存在量词 ($\exists$)，另外无量词 (quantifier-free, QF) 公式也是 SMT 的研究内容，本文主要关注无量词的 SMT 公式；
2. 谓词和函数用于描述个体变元的性质和个体间的映射关系；
3. 项 (term) 由如下规则递归定义：公式中所有变量、常量是项；若 f 是 n 元函数，且每个参数 t_i 均为项，则函数表示式 $f(t_1, \dots, t_n)$ 也是项。

此外 SMT 还在 SAT 的基础上加入了背景理论，这是让 SMT 具有更强表达能力的关键。对于一个 SMT 公式 F 和一个背景理论 T ，若存在一种赋值方案 α 使得 F 和 T 能同时满足，则称 F 是 T -可满足的， α 就是该 SMT 公式 F 的解，也被称为理论模型 (model)，记作 $\alpha \models F$ 。因此 SMT 问题可描述为判断 SMT 公式是否是 T -可满足的，并等价于判断 F 是否存在一个理论模型^[5]。例如下面的 SMT 公式：

$$\text{例 1 } (x + y \leq 10) \wedge (x \geq 5)$$

例 1 的命题逻辑形式是 $f_1 \wedge f_2$ ，其中的命题变量 f_1 和 f_2 在线性算术背景理论下解释为两个不等式 $x + y \leq 10$ 和 $x \geq 5$ ，能否找到 x 和 y 的某种赋值方式使得两个不等式同时成立就是要求解的问题。SMT 的背景理论包括：

1. 线性算术理论，包括基于整数集的线性算术 (Linear Integer Arithmetic, LIA) 和基于实数集的线性算术 (Linear Real Arithmetic, LRA)，以及它们的混合。该种理论公式表现为不等式或等式，其中的变量数据类型是整数或者实数，符号只包含 $+$, $-$, $=$, $\neq$, $\leq$, $\geq$ ，例如 $(x + y \leq 5) \wedge (x - z = 10)$ ；

2. 非线性算术理论，该理论是线性算术理论的拓展和延伸，同样基于整数集 (Non-Linear Integer Arithmetic, NIA) 或实数集 (Non-Linear Real Arithmetic, NRA)，包含了乘、除、取余等非线性操作，可用于表述任意形式的数学表达式^[2]；
3. 差分逻辑理论，该理论是线性算术的一部分，包括整数差分逻辑 (Integer Difference Logic, IDL) 和实数差分逻辑 (Rational Difference Logic, RDL)，形式表现为两个变量的差值等于或小于等于常量，例如 $(x - y \leq 3) \wedge (y - z = 10)$ 。差分逻辑的作用在于可以将 SMT 公式建模为以顶点表示变量、带权有向边表示约束条件的加权有向图，例如 $x - y \leq 3$ 表示 y 到 x 有权重为 3 的有向边，再通过检查该图是否存在权重之和为负的回路来高效地判断 SMT 公式的可满足性，若不存在则说明公式可满足，否则说明不可满足^[4,23]；
4. 数组理论 (Arrays, A)，用于表示高级编程语言中数组的相关操作，如 $read(a, i)$ 表示读取数组 a 的第 i 个位置的元素， $write(a, i, v)$ 表示将数组 a 第 i 个位置的元素设置为 v ；
5. 位向量理论 (Bit Vectors, BV)，用于表示位向量中的各种位运算操作，例如取反、与、或、异或、左移和右移等；
6. 未解释函数理论 (Uninterpreted Functions, UF)，用于表示还没有经过解释的抽象函数；
7. 字符串等其他理论。

此外，由于实际问题的复杂和多元性，SMT 通常会面临需要同时处理多种背景理论的情况，因此产生了将多种背景理论组合在一起的理论，称为组合理论，例如数组、未解释函数、整数线性算术的组合理论 (AUFLIA)，并出现了处理组合理论的判定方法，包括 Nelson-Oppen (NO) 方法^[24]，Delayed Theory Combination (DTC) 方法^[25]和 Ackerman 方法^[26]，这些方法也被广泛用于 SMT 求解技术中。

2.2 浮点数背景理论

本文关注的背景理论是浮点数背景理论 (Floating Point, FP)，该理论包括多种精度的浮点数类型以及各种操作。SMT-LIB 基于 IEEE 754-2008 标准^[27]对浮

点数制定的格式和运算来表示浮点数和相关操作^[21]，主要表示规则如下：

1. 浮点数类型表示为 Float16, Float32, Float64 和 Float128, 分别对应 IEEE binary16, binary32, binary64 和 binary128 形式。Float32 和 Float64 即为 32 位单精度和 64 位双精度的浮点数类型；
2. 具体数值用二进制位向量的形式表示，以及用 NaN 和 $\pm\infty$ 表示非数和正负无穷，例如：

例 2 (fp #b1 #b011111110 #b111111111111111111111111)

- 3 个 #b 开头的二进制位向量分别表示浮点数的符号位 (Sign)、阶码 (Exponent)、尾数 (Mantissa)，例 2 表示的浮点数数值为 -0.9999999403953552；
3. 运算操作主要包括：abs (取绝对值)，neg (取负)，add (加)，sub (减)，mul (乘)，div (除)，fma (融合乘加，如 $x * y + z$)，sqrt (取平方根)，rem (取余)，min/max (取最小/大值)，leq (小于等于) 等大小关系判断，isPositive (是正数) 等数值特征判断；
 4. 由于浮点数是计算机用于近似表示现实中某实数的数据类型，所以在运算中还需要规定舍入模式 (rounding mode)，在 SMT-LIB 中包括 RNE (向最近的偶数舍入)，RTP (向正无穷舍入)，RTZ (向零舍入) 等模式。

下面是一个简单的 SMT-LIB 格式的无量词浮点数约束 (QF_FP) 示例：

Listing 2.1: QF_FP 示例

```
1 (declare-fun a () Float64)
2 (declare-fun b () Float64)
3 (declare-fun c () Float64)
4 (assert (fp.eq (fp.add RNE a b) (fp.add RNE b c)))
5 (check-sat)
```

Listing 2.1 首先声明了 3 个 Float64 类型的浮点数变量 a, b, c，然后提出约束 $a + b = b + c$ ，相加时舍入模式为 RNE，最后判断可满足性。

2.3 优化问题及其算法

优化问题一般指寻找最优解的数学问题，本文关注 XSat 和 goSAT 将 SMT 求解问题转换为求目标函数（objective function）最小值点的优化问题，该优化问题中的目标函数定义如下^[6-7]：

定义 2.3.1 给定一个无量词浮点数约束公式 $c(x)$, $x \in FP_n$, $c(x)$ 可对应为一个目标函数 $f(x)$, $f(x)$ 满足如下规则：

1. $f(x)$ 非负，即 $\forall x \in FP_n, f(x) \geq 0$ ；
2. $f(x)$ 的所有零点等价于 $c(x)$ 的所有解，即 $f(\alpha) = 0 \Leftrightarrow \alpha \models c(x)$ 。

从定义 2.3.1 可以看出，若能找到目标函数的最小值点，即解决这个优化问题，就能得到对应 SMT 公式的可解性。本文采用 Python 的 SciPy 库^[28]中的 `scipy.optimize.fmin_cg`¹（简称 `fmin_cg`）求解目标函数的最小值点。`fmin_cg` 实现了一种较快的梯度下降算法：共轭梯度算法（Conjugate Gradient, CG）^[29]，其基本思路是从初始猜测点（initial guess）出发，沿着负梯度方向做线性搜索迭代到该方向上的最小值点，再利用梯度等信息计算出新的共轭的下降方向和步长，反复迭代直到得到函数的最小值点。`fmin_cg` 在目标函数只有全局最小值和选取的初始猜测点接近最小值点时会有更好的表现，所以更适用于局部优化（local optimization）^[28]。

2.4 启发式策略

启发式策略（heuristic strategy）是指基于直观感受和实际经验帮助求解优化问题实例的方法，不能保证找到最优解，但通常可以找到较优解或者可行解。一些基于启发式策略思想的算法框架目前已得到广泛运用，例如爬山算法、模拟退火算法、基因算法、进化策略、粒子群优化算法等^[30]。这类算法的特点是模仿自然界中的现象规律，例如基因算法模仿生物进化原理，将基因复制、交叉、变异等操作用于筛选和生成候选解，经过多轮“进化”来寻找最优解；又例如粒子群算法模仿自然界中鸟类等动物的群体行为，设立多个粒子，让各个粒子根据自身和群体的信息进行搜索，从而寻找最优解。本文主要针对目标函数最小

1 https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.fmin_cg.html

值点的实际求解过程提出帮助选取初始猜测点的启发式策略，从而使得 `fmin_cg` 能更好地求解。

第三章 SMT 求解器的评估

目前已有许多研究人员针对浮点数背景理论的 SMT 问题研究求解技术和开发求解器，为了明确当前的 SMT 求解器对浮点数约束问题的性能表现和能力范围，了解 SMT 求解器在浮点数约束问题中遇到的困难，探索进一步提升 SMT 求解器对浮点数约束问题求解效果的改进方法，本文选取了 4 个 SMT 求解器和 1 个 SMT-LIB 格式的无量词浮点数约束测试数据集（QF_FP benchmarks）进行评估和分析。

3.1 待评估的 SMT 求解器

3.1.1 Z3

Z3 是微软为解决软件验证和软件分析等领域的问题而研发的支持多种背景理论的 SMT 求解器，目前已经在工业领域得到应用，例如集成在 Spec#/Boogie3、Pex、HAVOC 等大型项目里^[3]。Z3 是当前表现最好的主流 SMT 求解器之一，具有成熟的求解技术和工具框架，在 SMT-COMP 也一直表现优秀，并且有良好的扩展性。图 3-1 是 Z3 的体系架构图。

Z3 接收输入的 SMT 公式后，先利用 Simplifier 对公式进行初步的简化，降低公式的复杂程度，然后使用 Compiler 将公式转换为由子句集和 congruence-closure nodes 组成的特殊数据结构。Z3 采用基于 DPLL 算法的 SAT 求解器对 SMT 公式的命题逻辑形式进行赋值，核心理论求解器 Congruence closure core 处理该赋值以及 SMT 公式中的未解释函数和等式。Theory Solvers 是处理线性算术、位向量、数组等理论的附属求解器（satellite solvers），帮助 Congruence closure core 处理这些背景理论及其组合。E-matching 则是用于处理量词的抽象机（abstract machine）。

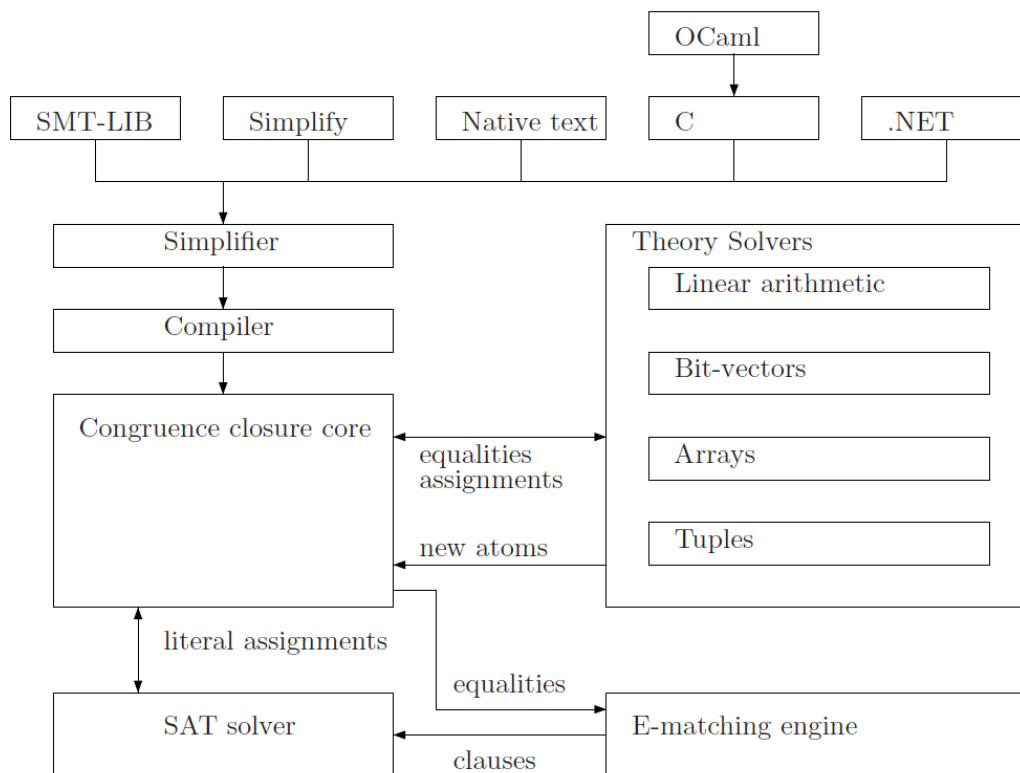


图 3-1 Z3 的体系架构

3.1.2 JFS

JFS 是 Liew 等人^[17]开发的主要求解浮点数约束的 SMT 求解器，将基于覆盖引导的模糊测试方法（coverage-guided fuzzing）运用在了浮点数约束求解问题中，其主要思想是将 SMT 公式转变为一个程序，该程序的输入对应 SMT 公式里的自由变量赋值，并把 SMT 公式里的每个约束都对应转变为一个 if-else 语句块，如果某个约束不满足就会在对应的 else 语句返回 0 退出，只有满足全部约束才能通过所有 if-else 语句块达到一个返回 1 的语句（称为 target），此时程序的输入就对应 SMT 公式的解。例如 2.2 节的 Listing 2.1 表示的 SMT 约束可以转变为如下的 C++ 程序：

Listing 3.1: JFS 程序示例

```

1 int FuzzOneInput(const uint8_t * data , size_t size){
2     double a = makeFloatFrom(data , size , 0, 63);
3     double b = makeFloatFrom(data , size , 64, 127);
4     double c = makeFloatFrom (data , size , 128, 191);

```

```

5     double a_plus_b = add_rne(a, b);
6     double b_plus_c = add_rne(b, c)
7     if (a_plus_b == b_plus_c) {} else return 0;
8     return 1; // TARGET REACHED
9 }

```

第 1 行的 FuzzOneInput 接收 size 字节的数据作为输入，第 2-4 行分别用 data 的第 0-63 位、64-127 位、128-192 位构造 64 位的浮点数变量 a, b, c，第 5-6 行得到 a 和 b、b 和 c 在 RNE 舍入模式的相加结果，第 7 行对应 Listing 2.1 第 4 行的约束，即判断 a + b 是否与 b + c 相等，如果相等则能通过该 if-else 语句块，到达第 8 行的 target 语句返回 1，否则返回 0。

JFS 利用基于覆盖引导的模糊测试工具（coverage-guided fuzzer）反复调用 FuzzOneInput，每次提供不同的输入，寻找能覆盖所有 if-else 语句块达到 target 的输入，从而找出 SMT 公式的解，或者达到资源限制停止寻找，因此 JFS 是 sound 和 incomplete 的，即找到的解是可以信任为一定正确的，但由于资源限制无法完全证明 SMT 公式是 unsat 的^[17]。

3.1.3 XSat

XSat 是 Fu 等人^[7]针对浮点数约束问题开发的 SMT 求解器，建立了浮点数约束可满足性问题与一类优化问题之间的等价关系，从而避免了直接对复杂的浮点数约束进行编码和推理。XSat 将浮点数约束公式转化为满足 2.3 节的定义 2.3.1 的目标函数 $f(x)$ ， $f(x)$ 的返回值表示输入 x 与约束的解之间的距离，当距离为 0 时就说明 x 是约束的解，若距离的最小值大于 0 则说明约束不可解。例如约束 $x \leq 4$ 可转变为如下的分段函数：

$$f(x) = \begin{cases} 0 & \text{if } x \leq 4 \\ (x - 4)^2 & \text{otherwise} \end{cases} \quad (3-1)$$

$f(x)$ 每个最小值点（即值为 0 的点）实际上就是 $x \leq 4$ 的每个解，因此约束求解问题被转变为了求目标函数的最小值点问题。XSat 采用马尔可夫链蒙特卡

洛方法 (Monte Carlo Markov Chain method, MCMC)^[31]求解函数的最小值点。在具体实现方面, XSat 先用一个 code generator 生成目标函数的 C 语言代码, 再用 Python 的 SciPy 库中的 `scipy.optimize.basinhopping`¹ (简称 `basinhopping`) 作为求函数最小值点的方式。basinhopping 实现了 MCMC 的一种变体: 盆地跳跃算法 (Basin Hopping, BH)^[32], 其基本思路是先从随机的初始位置 x_0 开始, 利用局部优化算法 (例如 2.3 节介绍的 `fmin_cg`) 得到 x_0 附近的极小值点 (即 “盆地”) 作为候选解, 再对 x_0 进行某种扰动, 从而 “跳跃” 到另一个位置 x_1 , 用同样的方式得到 x_1 附近的极小值点 (即新的 “盆地”) 作为候选解, 经过反复迭代后得到一系列候选解, 从中选出最小的作为全局最小值点。该算法的优势在于可以利用 “跳跃” 避免被困在局部的最小值点。

XSat 与 JFS 一样都是 `sound` 和 `incomplete` 的。因为目标函数是非负的, 如果能得到目标函数的最小值为 0 则说明对应的 SMT 公式一定是 `sat` 的, 但仍可能存在如下情况: 目标函数存在为 0 的最小值点, 但 XSat 在求解时得到了错误的大于 0 的 “最小值点”, 从而将实际是 `sat` 的 SMT 公式报告为 `unsat`, 因此 XSat 无法完全证明 SMT 公式是 `unsat` 的。XSat 尚未支持所有浮点数背景理论的操作, 目前仅支持浮点数背景理论的算术操作, 如 `fp.add` (加), `fp.mult` (乘), `fp.leq` (小于等于) 等, 但不支持 `fp.isNormal` (是常数, 即不是无穷或 NaN) 等其他操作。

3.1.4 goSAT

goSAT 是 Khadra 等人^[6]对 XSat 进行改进和再实现的 SMT 求解器, 其基本原理和特点与 Xsat 相同, 都将浮点数约束问题转变为求目标函数最小值的优化问题, 但 goSAT 采用即时编译方法 (Just-in-Time (JIT) compilation) 简化了将约束公式转化为目标函数的步骤, 使用起来比 XSat 更加方便, 还采用了功能更丰富的优化工具 NLOpt^[33]求解目标函数的最小值点, 能解决的问题范围比 BH 算法更广。

1 <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.basinhopping.html#scipy.optimize.basinhopping>

3.2 测试数据集

本文选取了 JFS 的作者用于测试 JFS 的数据集中的 QF_FP 部分¹作为用于评估的测试数据集 (benchmarks)。该 benchmarks 部分来源于 SMT-COMP 的竞赛测试集, 覆盖了浮点数背景理论的大部分语义操作, 如: fp.lt (小于), fp.leq (小于等于), fp.gt (大于), fp.geq (大于等于), fp.eq (等于), fp.add (加), fp.sub (减), fp.mul (乘), fp.div (除), fp.neg (取负), fp.isNaN (非数), fp.isInfinite (无穷), fp.isZero (零值), fp.isNormal (是常数) 等浮点数操作, 以及 and, or, not 等其他操作。

JFS 的作者还基于 JFS 不能完全证明 unsat 的局限性对该 benchmarks 进行了筛选, 只保留了 sat 和 unknown (可满足性未知, 这类情况 SMT 求解器一般会求解超时) 的部分, 而 XSat 和 goSAT 同样不能完全证明 unsat, 但 Z3 可以。如果用于测试的 benchmarks 中存在 unsat 的部分, 将对 JFS, XSat 和 goSAT 的评估带来不利的影响因素, 因此该 benchmarks 能更公平地测试 Z3, JFS, XSat 和 goSAT 的求解能力。

在 5.2.2 节将对该 benchmarks 进行具体分析。

3.3 评估方法

为了评估当前 SMT 求解器在浮点数约束问题中的性能表现, 本文设计评估方法流程如下:

1. 选取 3.1 节介绍的 Z3, JFS, XSat 和 goSAT 作为待评估的 SMT 求解器, 并安装这些求解器公开发布的稳定版本^{2, 3, 4, 5};
2. 选取 3.2 节介绍的 benchmarks, 分析和记录其特征;
3. 在相同的实验环境中让 Z3, JFS, XSat 和 goSAT 分别对 benchmarks 进行求解, 记录它们花费的时间和给出的判断结果;

¹ https://github.com/mc-imperial/jfs-fse-2019-artifact/tree/master/data/benchmarks/3-stratified-random-sampling/benchmarks/QF_FP

² <https://github.com/Z3Prover/z3>

³ <https://github.com/mc-imperial/jfs>

⁴ <https://github.com/zhoulaihu/xsat>

⁵ <https://github.com/abenkhadra/gosat>

4. 分析实验结果，评估 SMT 求解器的性能表现。

第四章 SMT 求解器的改进研究

第三章介绍的 XSat 将浮点数约束问题转化为求目标函数最小值点的优化问题进行求解，这一思路使得 XSat 具有较快的求解速度（第五章的实验结果将表明这一点），在了解其具体实现时，我们发现了一些可改进之处。本章将介绍 XSat 可改进的地方以及我们对此提出和实现的改进方法。

4.1 XSat 可改进的地方

XSat 在具体的代码实现¹里采用 3.1.3 节介绍的 `basinhopping` 求解目标函数的最小值点，在此过程中会使用局部优化方法（例如 2.3 节介绍的 `fmin_cg`）得到局部的最小值点。而 `fmin_cg` 这类方法在选取的初始猜测点质量较高，即接近最小值点时能更快、更准确地成功求解^[28]，但 XSat 并未对这方面进行专门的处理，只进行了较为随意的选择。因此我们认为可以在选取初始猜测点的方式上进行改进，让 `fmin_cg` 使用更高质量的初始猜测点进行求解，从而提升求解效果。

4.2 改进方法与实现

我们提出并实现了 2 个基于启发式策略的选取初始猜测点的改进方法：比例过滤和迭代更新。我们使用 Python 编写具体代码，在开始求解前会随机生成一定数目的初始猜测点，并对这些初始猜测点进行改进，最后调用 `fmin_cg` 使用改进后的初始猜测点求解目标函数的最小值。

4.2.1 目标函数的代码形式

在介绍改进方法前，首先介绍我们的目标函数的代码形式。一个 SMT 问题通常会包含许多具体约束，对应目标函数的代码则会表现为有许多 `if-else` 语句块，例如 2.1.2 节的 SMT 公式例 1 对应目标函数的 Python 代码如下：

1 <https://github.com/zhoulaiFu/xsat/blob/master/xsat.py#L125>

Listing 4.1: 目标函数示例

```
1 def objective_function(x0):
2     x, y = x0
3     segment0 = None
4     if (x + y) <= 10:
5         segment0 = 0.0
6     else:
7         segment0 = ((x + y) - 10) ** 2
8     segment1 = None
9     if x >= 5:
10        segment1 = 0.0
11    else:
12        segment1 = (5 - x) ** 2
13    return segment0 + segment1
```

第 3 和 8 行的 `segment0` 和 `segment1` 变量分别对应输入 x 和 y 与约束 $x + y \leq 10$ 和 $x \geq 5$ 的解之间的距离值，当 x 和 y 满足某个约束时，对应的 `segment` 变量就赋值为 0，否则将赋值为非负的表达式（第 4-7，9-12 行的 if-else 语句块），最后返回 `segment0` 与 `segment1` 的和。当这两个变量都为 0，即两个约束都满足时，目标函数的返回值才会是 0，此时 x 和 y 的值就是例 1 的解。

4.2.2 比例过滤

记初始猜测点为 x_0 ，如果 x_0 能满足 SMT 公式中的所有约束就相当于直接找到了该 SMT 问题的解，因此在直观感受上，如果 x_0 满足的约束数目越多，就越可能接近真正的解（最小值点），`fmin_cg` 的求解效果就可能越好。另一方面，由于 `fmin_cg` 在运行时需要多轮迭代反复调用目标函数计算函数值以及进行其他操作和处理，所以其开销远大于只调用一次目标函数的开销。基于这两个方面，我们提出了如下方法流程寻找能尽量满足更多约束数目的 x_0 提供给 `fmin_cg`：

1. 根据目标函数的复杂程度设置 x_0 需要预先满足的约束数目的比例 *percentage*，并改写目标函数的代码形式（可见后文的 Listing 4.2）；

2. 将 x_0 作为输入调用一次目标函数，并对 x_0 满足的约束数目计数，判断 x_0 满足的约束数目占总数的比例是否小于 *percentage*;
3. 如果 x_0 满足的约束数目占总数的比例小于 *percentage*，则目标函数返回一个特殊值表示直接过滤掉该 x_0 ，不进入 *fmin_cg* 求解，反之目标函数和该 x_0 将进入 *fmin_cg* 求解。

相比于直接使用 *fmin_cg* 求解，该方法额外花费了调用一次目标函数的开销，但能过滤掉低质量的 x_0 ，避免 *fmin_cg* 使用低质量的 x_0 花费大量时间求解，从总体上降低开销和提升求解效果。

我们基于如下思路粗略地衡量目标函数的复杂程度：一个具体约束会涉及到多个变量，该约束其实只要求涉及到的变量在某个取值范围内，对其他变量则不做任何要求。因此我们认为约束涉及的变量种数越多该约束就越复杂，进而可以使用所有约束的复杂程度衡量目标函数的总体复杂程度。具体采用如下计算策略：

1. 记目标函数的变量个数为 *var_n*，约束个数为 *cons_n*，每个约束涉及到的变量种数之和为 *cons_var_n*（如 $x + x + y$ 和 $x - y$ 均涉及 2 种变量，则 $cons_var_n = 2 + 2 = 4$ ）;
2. 此时可计算出平均每条约束涉及到的变量种数 $\frac{cons_var_n}{cons_n}$ ，进而得到所有变量中平均有 $\frac{cons_var_n}{cons_n * var_n}$ 比值的变量出现在每条约束里；
3. 该比值越高，说明目标函数越复杂，反之则越不复杂。

目标函数越复杂， x_0 满足约束的数目可能就越少，因此 *percentage* 应该设置低一些，否则绝大部分 x_0 都会被直接过滤掉，很容易错失原本能成功求解的 x_0 ；反之则应该设置得高一些，让 x_0 尽量满足更多的约束，起到过滤的效果。因此实际的 *percentage* 应该由

$$1 - \frac{cons_var_n}{cons_n * var_n} \quad (4-1)$$

得到。此外，我们设置了一个区间 $[low, high]$ 作为 *percentage* 的取值范围，避免出现 *percentage* 过高或过低的极端情况，影响到实际效果，根据实际测试中的经验总结，该区间的数值设置为 $[0.3, 0.5]$ 。综上，*percentage* 的计算公式如等式 4-2 所示：

$$percentage = (1 - \frac{cons_var_n}{cons_n * var_n}) * (high - low) + low \quad (4-2)$$

采用比例过滤的方法后，目标函数的代码形式将在 Listing 4.1 的基础上添加对 x_0 满足的约束数目计数和根据设置的比例判断是否要过滤 x_0 的部分，如 Listing 4.2 所示：

Listing 4.2: 改写后的目标函数示例

```

1 def new_objective_function(x0):
2     count = 0
3     x, y = x0
4     segment0 = None
5     if (x + y) <= 10:
6         count += 1
7         segment0 = 0.0
8     else:
9         segment0 = ((x + y) - 10) ** 2
10    segment1 = None
11    if x >= 5:
12        count += 1
13        segment1 = 0.0
14    else:
15        segment1 = (5 - x) ** 2
16    if count < 2 * 0.35: # cons_n * percentage
17        return 1e10
18    return segment0 + segment1

```

第 2 行设置了计数变量 `count`，并在第 6 和 12 行插入了因为满足约束而进行的计数操作，在第 16 和 17 行对 x_0 满足的约束比例进行判断（该目标函数有 2 个变量，和 2 个分别涉及 2 种、1 种变量的约束，因此根据公式 4-2 可得 $percentage = (1 - \frac{2+1}{2 \times 2}) \times (0.5 - 0.3) + 0.3 = 0.35$ ），若小于设置的比例则返回特殊

值 $1e10$ 表示 x_0 应该被过滤掉，否则正常返回。

图 4-1 表示了比例过滤方法的运行流程。

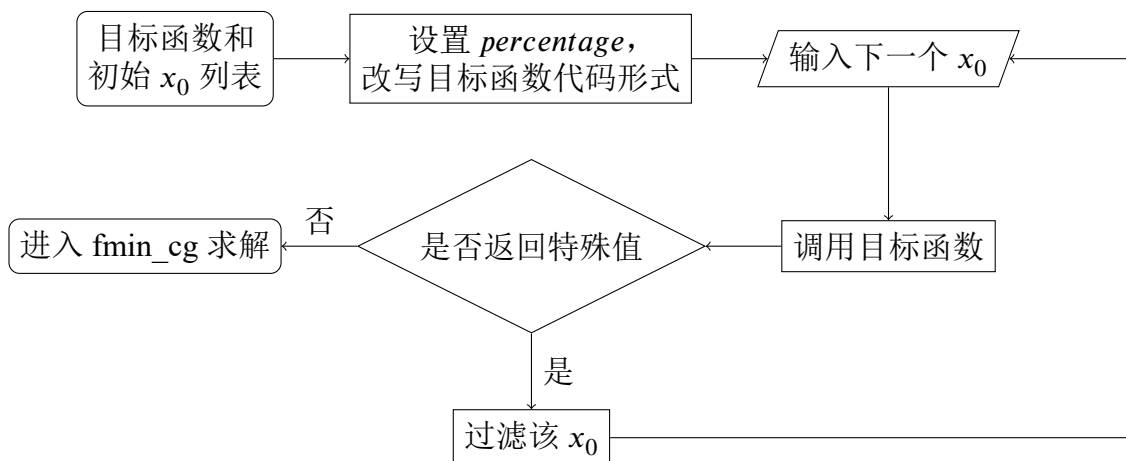


图 4-1 比例过滤的运行流程

4.2.3 迭代更新

`fmin_cg` 有时会因为某些原因（例如无法得到合适的下降步长等）不能正确迭代到目标函数的最小值点，此时 `fmin_cg` 会在最后停止迭代的位置 x_{exit} 退出。但根据 2.3 节介绍的梯度下降算法的基本流程可以知道，虽然 x_{exit} 不是真正的最小值点，但相比于一开始的 x_0 ，进行了下降操作的 x_{exit} 应该是更接近最小值点的。基于这一思路，我们提出了如下方法帮助 `fmin_cg` 对求解失败的目标函数进行重新求解：

1. 获取上一次调用 `fmin_cg` 求解失败时得到的 x_{exit} ；
2. 在 x_{exit} 附近选取新的 x_0 提供给下一次 `fmin_cg` 的求解；
3. 重复以上步骤，直到成功求解或达到资源限制。

在实际实现中，我们设置了一个值 d 作为 x_{exit} 附近的搜索范围，即在区间 $[x_{exit} - d, x_{exit} + d]$ 内选取 x_0 作为新的初始猜测点，而不是重新随机地选取 x_0 。根据实际的测试表现，我们设置 d 的值为 100。

图 4-2 表示了迭代更新方法的运行流程。

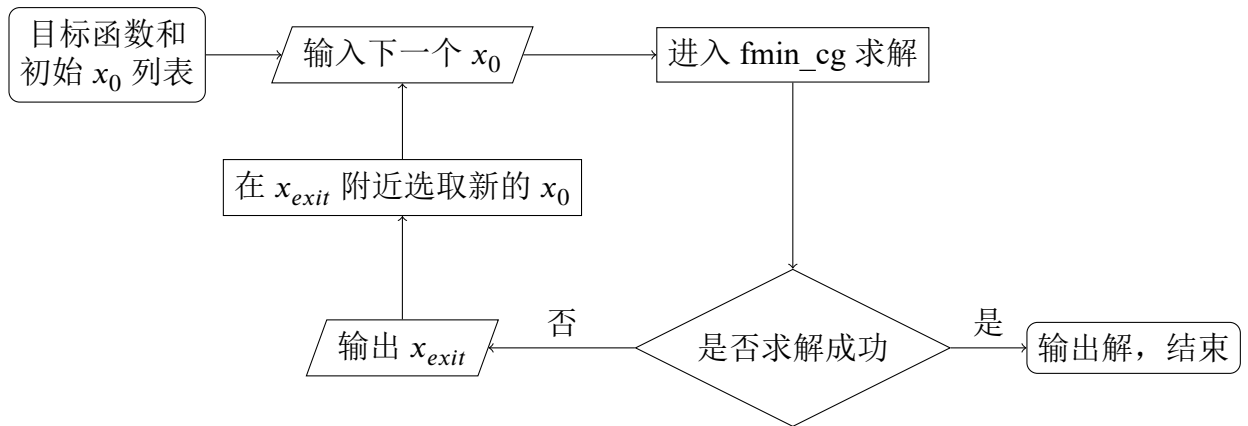


图 4-2 迭代更新的运行流程

4.2.4 两个方法的组合

上述两个方法从不同的方面对 x_0 的选取进行了改进，在实际运用中它们可以组合起来共同发挥作用，组合后的完整步骤如下：

1. 随机生成一定数目的初始 x_0 ；
2. 根据目标函数的复杂程度设置 x_0 需要预先满足的约束数目的比例 *percentage*，并改写目标函数的代码形式；
3. 将当前 x_0 作为输入调用一次目标函数，并对该 x_0 满足的约束数目计数，判断该 x_0 满足的约束数目占总数的比例是否小于 *percentage*；
4. 如果该 x_0 满足的约束数目占总数的比例小于 *percentage*，则目标函数返回一个特殊值表示需要过滤掉该 x_0 ，不进入 fmin_cg 求解，直接处理下一个 x_0 ，反之将进入 fmin_cg 求解；
5. 若当前 x_0 未被过滤进入了 fmin_cg 求解，则查看其求解结果：若求解成功，则直接输出解结束求解，否则输出 fmin_cg 最后停止迭代的位置 x_{exit} ；
6. 若 fmin_cg 未求解成功，在 x_{exit} 附近选取新的 x_0 作为之后输入的 x_0 ，重复上述第 3 到 5 步，直到求解成功或达到资源限制。

图 4-3 表示了两个方法组合之后的运行流程。

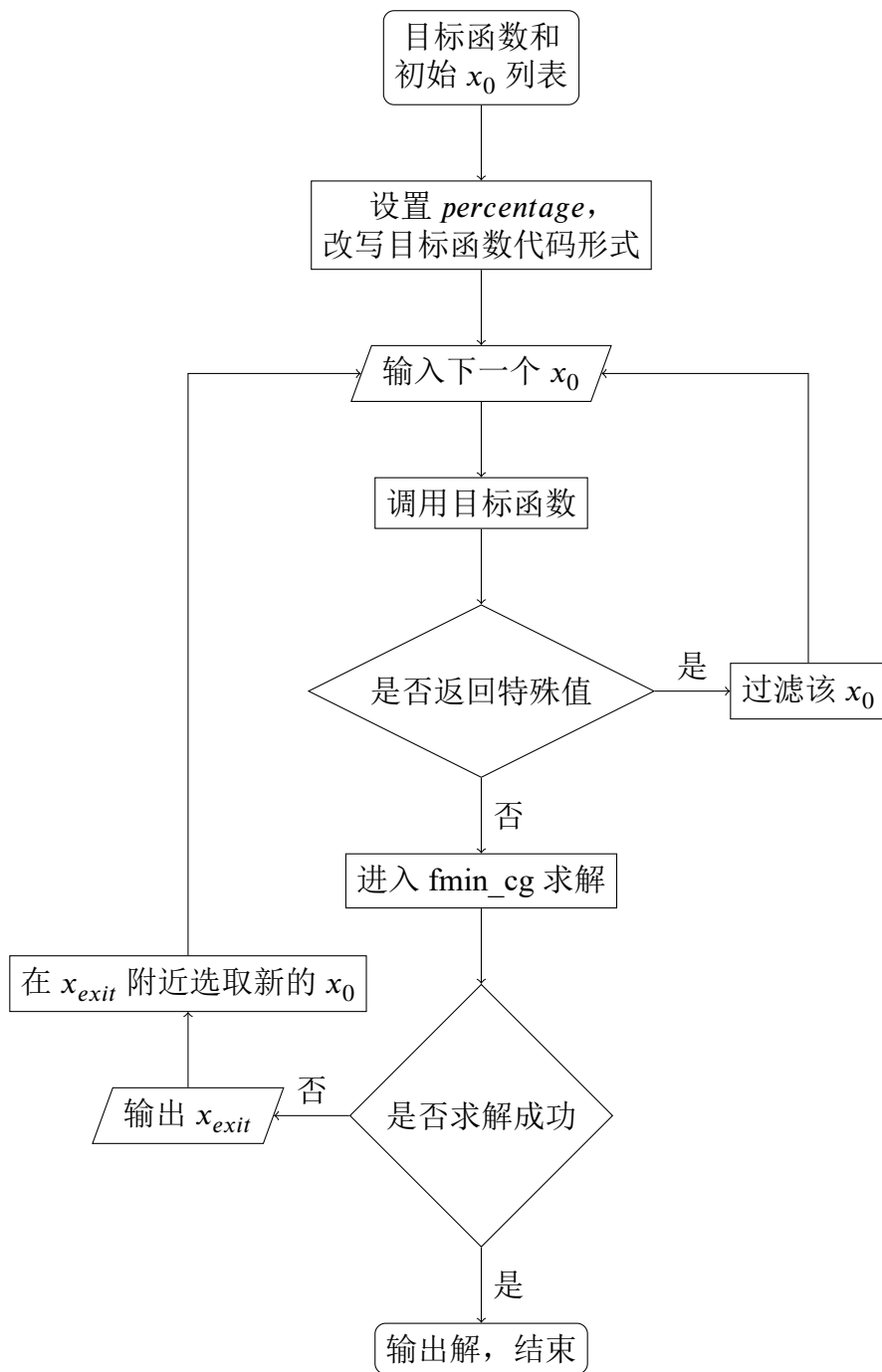


图 4-3 比例过滤和迭代更新组合后的运行流程

第五章 实验分析

本章将介绍对选取的 SMT 求解器：Z3，JFS，XSat 和 goSAT 进行性能评估的具体操作和结果，同时分析和总结了它们的表现，并设计了 4 组实验测试第四章提出的两种改进方法及其组合的实际效果，证明这两种方法能为求解目标函数的最小值带来帮助，从而能实现对 XSat 的改进。

5.1 实验环境

本文的实验在表 5-1 所示的实验环境中进行，同时展示了实现改进方法时所用的 Python 及相关库：SciPy 和 Autograd 的版本。

表 5-1 实验环境

操作系统	Ubuntu 18.04.6 LTS
CPU	Intel(R) Xeon(R) Gold 5117 CPU @ 2.00GHz
RAM	64GB
Python 版本	Python 3.9.0
SciPy 版本	scipy 1.13.0
Autograd 版本	autograd 1.6.2

5.2 评估分析

本节将详细介绍对 Z3，JFS，XSat 和 goSAT 进行性能评估的具体操作，展示评估结果并进行分析。

5.2.1 SMT 求解器配置

目前 JFS, XSat 和 goSAT 都只公开发布了一个稳定版本, 但 Z3 是一个不断更新的、发布过很多版本的 SMT 求解器, 我们在进行实验时选取了 Z3 当时发布的最新版本, 表 5-2 中展示了本文选取的 SMT 求解器及其版本信息。

表 5-2 SMT 求解器的版本

选取的 SMT 求解器	版本信息
Z3	version 4.12.3 - 64 bit - build hashcode 19e9212
JFS	0.0.0.0(d38b368) (fse_2019_paper_version-3-gd38b368)
XSat	version 12/18/2015
goSAT	v0.1

5.2.2 测试数据集分析

我们选取在 3.2 节介绍的测试数据集 (benchmarks) 对 SMT 求解器进行评估。该 benchmarks 共有 160 个 SMT-LIB 格式的 smt2 文件, 每个文件代表一个 QF_FP 理论的 SMT 问题, 其中声明了变量和具体约束。为了判断对应 SMT 问题的复杂程度从而比较 SMT 求解器的求解能力和范围, 本文统计了该 benchmarks 每个 smt2 文件中 declare-fun 语句 (如 Listing 2.1 第 1-3 行) 的个数, 得到该 benchmarks 中每个 smt2 文件声明的变量数目 var_n , 并以此大致判断对应 SMT 问题的复杂程度, 即 var_n 越大对应的 SMT 问题越复杂。该 benchmarks 根据 var_n 的范围可分为以下 3 部分:

1. $var_n < 10$: 共 102 个 smt2 文件;
2. $10 \leq var_n \leq 50$: 共 37 个 smt2 文件;
3. $var_n > 50$: 共 21 个 smt2 文件。

在统计和分析选取的 SMT 求解器的评估结果时, 我们统计了全部的结果和按以上 3 部分 var_n 分类后的结果, 并分别进行分析。另外, 正如 3.2 节所介绍的, 该 benchmarks 不包含 unsat 的部分, 只包含 sat 或 unknown, 因此在评估的时候我们将主要以 SMT 求解器求解出 sat 的数目作为 SMT 求解器的评估指标之一。

5.2.3 评估结果与分析

我们按照在 3.3 节介绍的评估方法流程对选取的 SMT 求解器进行评估，编写了 Shell 脚本进行对 benchmarks 的分析和对 SMT 求解器的使用，并获取和统计求解信息。

在使用选取的 SMT 求解器对 benchmarks 求解时，我们采用其默认的工作模式，并设置超时时间为 900 秒，利用 Linux 的 timeout 命令限制 SMT 求解器的求解时间。我们根据使用 SMT 求解器进行求解的执行命令的返回值判断是顺利求解完毕（返回值为 0）还是发生了超时（返回值为 124）或者报错（返回值为其他非零值），并获取 SMT 求解器的求解时间和报告信息，记录在 csv 文件里。

Z3, JFS, XSat, goSAT 在选取的 benchmarks 上的全部求解结果如表 5-3 所示，根据变量数目 var_n 分为 3 部分后的结果分别如表 5-4，表 5-5，表 5-6 所示。表的第 1 列是 SMT 求解器的名称，第 2 到 5 列是求解结果，分别是：求解为 sat，求解为 unsat 或 unknown，求解超时（timeout）和报错（error），第 6 列是对 benchmarks 求解花费的平均时间（单位秒，包括 timeout 和 error 的部分）。由于 goSAT 将不能判断为 sat 的部分全部报告为 unknown，而其他 3 个 SMT 求解器则会报告 unsat，并且我们更关注求解为 sat 的数目，因此我们将 unsat 和 unknown 的数目放在一起记录。

表 5-3 总体求解结果

SMT 求解器	sat	unsat/unknown	timeout	error	平均求解时间（秒）
Z3	89	0	71	0	464.708
JFS	87	0	73	0	420.427
XSat	100	24	5	31	42.497
goSAT	84	51	0	25	4.217

表 5-4 $var_n < 10$ 部分的求解结果

SMT 求解器	sat	unsat/unknown	timeout	error	平均求解时间（秒）
Z3	72	0	30	0	344.852
JFS	72	0	30	0	274.408
XSat	63	14	0	25	1.276
goSAT	61	20	0	21	0.066

表 5-5 $10 \leq var_n \leq 50$ 部分的求解结果

SMT 求解器	sat	unsat/unknown	timeout	error	平均求解时间（秒）
Z3	16	0	21	0	561.198
JFS	14	0	23	0	565.134
XSat	28	3	0	6	3.703
goSAT	20	13	0	4	0.616

表 5-6 $var_n > 50$ 部分的求解结果

SMT 求解器	sat	unsat/unknown	timeout	error	平均求解时间（秒）
Z3	1	0	20	0	876.855
JFS	1	0	20	0	874.706
XSat	9	7	5	0	311.062
goSAT	3	18	0	0	30.719

我们根据表 5-3 展示的总体结果从以下方面进行分析：

1. 在求解时间方面，Z3 花费的时间最久，有 44.38% 的 benchmarks 求解超时；JFS 的求解时间略少于 Z3，但仍属于比较久的程度，并且有 45.63% 的 benchmarks 超时；而 XSat 和 goSAT 花费的时间则极大地少于 Z3 和 JFS，到达了数量级的减少，timeout 的数目也很少或者没有。另外，goSAT 作为在 XSat 的基础上改进而来的 SMT 求解器，求解时间也相对于 XSat 有很大提升。
从表 5-3 的结果可得，XSat 花费的时间比 Z3 少 90.86%，比 JFS 少 89.89%，而 goSAT 花费的时间比 Z3 少 99.09%，比 JFS 少 99.00%，比 XSat 少 90.08%；
2. 在求解数目方面，Z3，JFS 和 goSAT 在总体上的表现相近，XSat 的表现最好，求解出了最多数目的 sat 结果，比 Z3 多 12.36%，比 JFS 多 14.94%，比 goSAT 多 19.05%；
3. 在求解器的稳定性方面，Z3 和 JFS 具有较好的稳定性，没有出现 error，也没有报告 unsat 或 unknown；而 XSat 和 goSAT 的稳定性则稍差，分别有 19.38% 和 15.62% 的 benchmarks 出现了 error，主要原因包括求解器自身程序的不完善和部分 benchmarks 中存在 XSat 和 goSAT 暂时不支持的浮点数操作，例如 fp.isZero（零值），fp.isNormal（是常数）等。此外 XSat 和 goSAT 还报告了不少 unsat/unknown，说明出现了如 3.1.3 节最后介绍的求

解错误的情况。

根据表 5-4，表 5-5 和表 5-6 展示的按照变量数目分类后的结果，我们分别进行了如下的分析：

1. 对于复杂程度较低的 SMT 问题，Z3 和 JFS 在求解数目和稳定性上的表现相同，但 JFS 的求解时间比 Z3 少 20.43%；XSat 和 goSAT 的求解数目比 Z3 和 JFS 少，并且出现了一些求解错误和报错的情况，但求解时间仍然提升很大：XSat 的求解数目比 Z3 和 JFS 少 12.50%，求解时间比 Z3 快 99.63%，比 JFS 快 99.53%；goSAT 的求解数目比 Z3 和 JFS 少 15.28%，求解时间比 Z3 和 JFS 快 99.98%，比 XSat 快 94.83%；
2. 对于复杂程度适中的 SMT 问题，JFS 在求解数目和时间上的表现接近但略低于 Z3，而 XSat 和 goSAT 的求解数目和时间则相比于 Z3 和 JFS 有很大提升：XSat 的求解数目比 Z3 多 75.00%，比 JFS 多 100.00%，求解时间比 Z3 和 JFS 少 99.34%；goSAT 的求解数目比 Z3 多 25.00%，比 JFS 多 42.86%，求解时间比 Z3 和 JFS 少 99.89%；XSat 的求解数目比 goSAT 多 40.00%，但 goSAT 的求解时间比 XSat 少 83.36%；另外，XSat 和 goSAT 仍出现了错误求解和报错的情况；
3. 对于复杂程度较高的 SMT 问题，Z3 和 JFS 都只求解出来了 1 个，绝大部分都超时了；XSat 和 goSAT 求解出来了更多个，但求解出错的情况也较多。在求解时间上，Z3 和 JFS 表现很接近（因为绝大部分都超时了），而 XSat 花费的时间比 Z3 少 64.53%，比 JFS 少 64.44%，goSAT 花费的时间比 Z3 少 96.50%，比 JFS 少 96.49%。

经过上面的分析，我们对选取的 SMT 求解器的性能评估总结如下：

1. 相比于直接对浮点数约束进行推理的主流 SMT 求解器，将浮点数约束问题转变为优化问题等其他问题进行求解的 SMT 求解器的求解能力更强，能在求解时间上得到很大的提升，在求解数目上也能有所增加；
2. 将浮点数约束问题转变为优化问题等其他问题进行求解的 SMT 求解器在稳定性上可能存在不足（这也和 SMT 求解器是否不断进行维护和更新有关），会出现错误求解和程序报错的情况；

3. 将浮点数约束问题转变为优化问题等其他问题进行求解的 SMT 求解器的能力范围比主流 SMT 求解器小，存在只能完全证明 `sat`、不能完全证明 `unsat`，以及不支持一些浮点数操作的问题。

5.3 改进研究

本节将详细介绍对第四章提出和实现的两种改进方法及其组合进行测试的实验设计，并展示改进效果。

5.3.1 实验设计

我们采用 Python 作为实验的编程语言，并从在 3.2 节介绍的 `benchmarks` 中的 160 个 SMT 约束问题里选取了 135 个转变为对应的目标函数，保存在 `py` 文件里，用于实现和测试两种改进方法，并记作 `original_objective_function`。正如 5.2.2 节的分析，该 `benchmarks` 不包含 `unsat` 的部分，所以我们统计对目标函数求得的最小值为 0 的数目作为成功求解的数目，以此来判断改进效果。

我们首先利用正则匹配等操作解析 `original_objective_function` 的代码文本，按照 4.2.2 节介绍的流程获取目标函数的变量个数、约束个数等用于计算过滤比例的数据，从而将 `original_objective_function` 的代码形式改写为如 Listing 4.2 所示的形式，并写入到了新的 `py` 文件里，记作 `new_objective_function`，用于实现和测试比例过滤方法。

然后我们为每个目标函数随机生成初始的输入 x_0 ，具体做法是根据每个目标函数定义的变量参数个数，在 $[-1000, 1000]$ 范围内随机生成浮点数数组，共生成 500 组，保存在 `txt` 文件里，即每个目标函数都有 500 组随机生成的不同的 x_0 用于之后的求解。

最后我们使用在 2.3 节介绍的 `fmin_cg` 依次求解所有目标函数的最小值，并对得到的最小值保留 9 位小数判断其是否为 0，进而判断是否求解成功，还设置了每个目标函数的求解总时间限制为 1800 秒。调用 `fmin_cg` 时需要提供目标函数的梯度作为参数，因此我们采用 Python 的 Autograd 库¹中的 `grad` 计算目标函数的梯度。

¹ <https://github.com/hips/autograd>

我们分不同情况设计了如下 4 组实验进行测试，并记录求解时间和成功数目：

1. 不采用任何改进方法，使用 `original_objective_function` 和对应的 500 组初始 x_0 进行测试。每个目标函数将和它的一组 x_0 进入 `fmin_cg` 进行求解，若解得的最小值不为 0 则再使用下一组 x_0 求解，直到求解成功，或者 500 组 x_0 全部都没能求解成功，或者达到总时间限制，如 Algorithm 1 所示；

Algorithm 1 不采用任何改进方法

Input: The *objective_function* and the list of x_0 .

Output: The minimum of *objective_function* is 0 if such a minimum can be found, or is not 0, or timeout.

```
while use fmin_cg solving the minimum of objective_function do
  get the next  $x_0$ 
  test_cg(objective_function,  $x_0$ )
  check the solving result and solving time
end while
```

2. 仅采用比例过滤方法，使用改写后的 `new_objective_function` 和对应的 500 组初始 x_0 进行测试。每个目标函数和它的一组 x_0 在进入 `fmin_cg` 进行求解前，该组 x_0 将先作为参数调用一次目标函数，若目标函数返回了特殊值 $1e10$ 则直接抛弃该组 x_0 ，继续尝试下一组 x_0 ，否则再进入 `fmin_cg` 进行求解，如 Algorithm 2 所示。将该组 x_0 将作为参数调用目标函数的时间也会被计算在求解总时间内；

Algorithm 2 比例过滤

Input: The *objective_function* and the list of x_0 .

Output: The minimum of *objective_function* is 0 if such a minimum can be found, or is not 0, or timeout.

```
while use fmin_cg solving the minimum of objective_function do
  get the next  $x_0$ 
  if objective_function( $x_0$ ) =  $1e10$  then
    continue
  else
    test_cg(objective_function,  $x_0$ )
    check the solving result and solving time
  end if
end while
```

3. 仅采用迭代更新方法，使用 `original_objective_function` 和对应的 500 组初始 x_0 中的前 50 组进行测试。每个目标函数将和它的一组 x_0 进入 `fmin_cg` 进行求解，并获取 `fmin_cg` 退出时所在的位置 x_{exit} ，若解得的最小值不为 0，则在 x_{exit} 附近，即 4.2.3 节介绍的搜索范围 $[x_{exit} - 100, x_{exit} + 100]$ 中随机生成新一组 x_0 补充到待测的 x_0 中，即第 i 组 x_0 将生成第 $50 + i$ 组 x_0 ，以 50 组作为一个轮次不断进行迭代更新，如 Algorithm 3 所示。最终的 x_0 组数将与另外 3 种情况的实验相同，都是 500 组（假设全部求解失败且未超时），从而确保比较的公平性；

Algorithm 3 迭代更新

Input: The *objective_function* and the list of x_0 .

Output: The minimum of *objective_function* is 0 if such a minimum can be found, or is not 0, or timeout.

```

while use fmin_cg solving the minimum of objective_function do
  get the next  $x_0$ 
   $x_{exit} \leftarrow \text{test\_cg}(\text{objective\_function}, x_0)$ 
   $\text{new\_}x_0 \leftarrow \text{random\_select}([x_{exit} - 100, x_{exit} + 100])$ 
  add  $\text{new\_}x_0$  to the list of  $x_0$ 
  check the solving result and solving time
end while

```

4. 同时采用比例过滤和迭代更新方法，使用改写后的 `new_objective_function` 和对应的 500 组初始 x_0 中的前 50 组进行测试。每个目标函数和它的一组 x_0 在进入 `fmin_cg` 进行求解前先判断是否会被过滤，若会被过滤则直接继续尝试下一组，若不会被过滤则再进入 `fmin_cg` 进行求解，并在 `fmin_cg` 退出时所在的位置附近生成新一组 x_0 ，如 Algorithm 4 所示。最终的 x_0 组数在全部求解失败且未超时的情况下同样会是 500 组，如果出现了初始的 x_0 和生成的新 x_0 都有很多被过滤导致尝试完所有 x_0 后没有成功求解且 x_0 不足 500 组，则会再在 $[-1000, 1000]$ 内随机生成一组 x_0 进行补充，直到求解成功或足够 500 组。

Algorithm 4 比例过滤 + 迭代更新

Input: The *objective_function* and the list of x_0 .

Output: The minimum of *objective_function* is 0 if such a minimum can be found, or is not 0, or timeout.

```
while use fmin_cg solving the minimum of objective_function do
  get the next  $x_0$ 
  if objective_function( $x_0$ ) = 1e10 then
    continue
  else
     $x_{exit} \leftarrow \text{test\_cg}(\text{objective\_function}, x_0)$ 
     $\text{new\_}x_0 \leftarrow \text{random\_select}([x_{exit} - 100, x_{exit} + 100])$ 
    add  $\text{new\_}x_0$  to the list of  $x_0$ 
    check the solving result and solving time
  end if
  if current  $x_0$  is the last one then
     $\text{new\_}x_0 \leftarrow \text{random\_select}([-1000, 1000])$ 
    add  $\text{new\_}x_0$  to the list of  $x_0$ 
  end if
end while
```

5.3.2 效果分析

5.3 节设计的 4 组实验的结果如表 5-7 所示，第 1 列是采用的改进方法（或不采用任何改进方法），第 2 列是该方法在所有目标函数里成功求解出最小值为 0 的数目，第 3 列是该方法求解完所有目标函数花费的平均时间（包括了超时的部分）。

表 5-7 4 组实验的求解结果

采用方法	成功求解数目	平均花费时间（秒）
不采用任何改进方法	99	365.91
比例过滤	100	337.32
迭代更新	100	350.44
比例过滤 + 迭代更新	101	323.33

采用改进方法相比于不采用任何改进方法，在成功求解数目和平均花费时间上的提升如表 5-8 所示，可以看出比例过滤和迭代更新都能为求解效果带来提升，当两种方法组合时效果最好。

表 5-8 改进效果

采用方法	成功求解数目增加	平均花费时间减少
比例过滤	1.01%	7.81%
迭代更新	1.01%	4.24%
比例过滤 + 迭代更新	2.02%	11.64%

我们分析了采用改进方法后多出的成功求解的目标函数的求解情况，如表 5-9 所示，griggio_fmcad12_gaussian_c_125 和 griggio_fmcad12_div2_c_10 是在不采用任何改进方法时未能成功求解，但分别在采用了比例过滤或迭代更新方法后能成功求解的目标函数，并且它们在采用了比例过滤和迭代更新的组合之后也能成功求解。

表 5-9 多出的成功求解的目标函数的求解情况

目标函数	采用方法	求解结果	花费时间（秒）	尝试 x_0 组数
griggio_fmcad12_gaussian_c_125	不采用任何改进方法	超时	1800.00	146
	比例过滤	成功	772.93	169
	迭代更新	超时	1800.00	308
	比例过滤 + 迭代更新	成功	576.88	136
griggio_fmcad12_div2_c_10	不采用任何改进方法	失败	307.60	500
	比例过滤	失败	235.52	500
	迭代更新	成功	86.52	171
	比例过滤 + 迭代更新	成功	69.45	164

可以看出，griggio_fmcad12_gaussian_c_125 采用了比例过滤方法后，在更短的时间内尝试了更多组的 x_0 ，从而成功求解；采用迭代更新方法后，虽然未能求解成功，但也尝试了更多组的 x_0 ，我们推测失败的原因是该情况对前 50 组 x_0 都进行迭代更新，导致花费时间较长，迭代不够深入，最终超时未能成功求解；采用比例过滤和迭代更新的组合后，效果提升比只采用比例过滤方法更明显，并且没有像只采用迭代更新方法那样超时，我们推测这是因为比例过滤方法保留了更高质量的 x_0 ，让迭代更新方法只使用这些更高质量的 x_0 进行更多轮次、更深入的迭代，因此取得了更好的效果。

griggio_fmcad12_div2_c_10 采用了比例过滤方法后使用更短的时间尝试完了 500 组 x_0 ，但因为这 500 组 x_0 均不能成功求解，所以求解失败；而采用了迭

代更新方法后则生成了能成功求解的 x_0 ，并在采用比例过滤和迭代更新的组合后花费的时间更短。

第六章 总结与展望

6.1 总结

本文研究了当前 SMT 求解器在浮点数约束问题上的性能表现和能力范围，具体选取了当前主流的 SMT 求解器 Z3 和针对浮点数约束问题的 JFS, XSat, goSAT 进行了评估分析，并得出了如下结论：相比于主流 SMT 求解器，将浮点数约束问题转变为优化问题等其他问题进行求解的 SMT 求解器在处理浮点数约束问题时具有更好的表现，能大幅度缩短求解时间和增加求解数目，但也存在稳定性不足和能力范围不够全面等问题。

本文还针对 XSat 的可改进之处提出并实现了两个改进方法：比例过滤和迭代更新，并设计了 4 组实验证明了这两种方法及其组合能提高初始猜测点的质量，有效缩短求解目标函数最小值点的时间，提升求解效果。

6.2 不足与展望

由于时间、精力和能力有限，本文的研究工作还存在不少不足之处，一些研究还处于比较简陋和不够深入的状态，希望在未来能做出改进和进行更加深入的思考与研究：

1. 选取的进行评估的 SMT 求解器数量较少，覆盖范围不够全面，例如在 1.2.2 节的相关工作中介绍的近年来表现较好的 CVC5 和 Bitwuzla, XSat 的作者设计的另一个工具 CoverMe^[34], He 等人基于随机局部搜索 (Stochastic Local Search, SLS) 实现的针对浮点数约束问题的 OL1V3R^[18]等，也都值得进行评估。选取的用于评估的测试数据集也可以从更多来源进行搜集，评估方法也还存在改进空间。
2. 提出的改进方法比较简陋，还存在缺陷，例如比例过滤方法仍然存在可能把原本能成功求解的初始猜测点过滤掉的问题，衡量目标函数复杂程度的

方式也较为简陋，未来可以考虑更全面和健壮的衡量策略，以及设置动态比例等过滤方式。

参考文献

- [1] 王戟, 詹乃军, 冯新宇, 等. 形式化方法概貌[J]. 软件学报, 2019, 30(01): 33-61. DOI: 10.13328/j.cnki.jos.005652.
- [2] 王翀, 吕荫润, 陈力, 等. SMT 求解技术的发展及最新应用研究综述[J]. 计算机研究与发展, 2017, 54(07): 1405-1425.
- [3] De MOURA L, BJØRNER N. Z3: An Efficient SMT Solver[C]//Tools and Algorithms for the Construction and Analysis of Systems. Springer Berlin Heidelberg: 337-340.
- [4] 唐傲, 王晓峰, 何飞. 可满足性模理论综述[J]. 计算机工程与科学, 2024, 46(03): 400-415.
- [5] 李婧. SMT 求解器技术对比分析及其能力扩展研究[D]. 2010.
- [6] KHADRA M A B, STOFFEL D, KUNZ W. GoSAT: Floating-point satisfiability as global optimization[C]//2017 Formal Methods in Computer Aided Design (FMCAD): 11-14. DOI: 10.23919/FMCAD.2017.8102235.
- [7] FU Z, SU Z. XSat: A Fast Floating-Point Satisfiability Solver[C]//Computer Aided Verification. Springer International Publishing: 187-209.
- [8] 金继伟, 马菲菲, 张健. SMT 求解技术简述[J]. 计算机科学与探索, 2015, 9(07): 769-780.
- [9] BARRETT C, TINELLI C. Satisfiability Modulo Theories[M]//CLARKE E M, HENZINGER T A, VEITH H, et al. Handbook of Model Checking. Cham: Springer International Publishing, 2018: 305-343. DOI: 10.1007/978-3-319-10575-8_11.
- [10] SEBASTIANI R. Lazy Satisfiability Modulo Theories[J]. Journal on Satisfiability, Boolean Modeling and Computation, 2007, 3: 141-224. DOI: 10.3233/SAT 190034.

- [11] NIEUWENHUIS R, OLIVERAS A, TINELLI C. Solving SAT and SAT Modulo Theories: From an abstract Davis–Putnam–Logemann–Loveland procedure to DPLL(T)[J]. J. ACM, 2006, 53(6): 937-977. DOI: 10.1145/1217856.1217859.
- [12] BARBOSA H, BARRETT C, BRAIN M, et al. Cvc5: A Versatile and Industrial-Strength SMT Solver[C] // Tools and Algorithms for the Construction and Analysis of Systems. Springer International Publishing: 415-442.
- [13] CIMATTI A, GRIGGIO A, SCHAAFSMA B J, et al. The MathSAT5 SMT Solver[C] // Tools and Algorithms for the Construction and Analysis of Systems. Springer Berlin Heidelberg: 93-107.
- [14] DUTERTRE B. Yices 2.2[C] // Computer Aided Verification. Springer International Publishing: 737-744.
- [15] NIEMETZ A, PREINER M, BIERE A. Boolector 2.0[J]. Journal on Satisfiability, Boolean Modeling and Computation, 2014, 9: 53-58. DOI: 10.3233/SAT190101.
- [16] NIEMETZ A, PREINER M. Bitwuzla[C] // Computer Aided Verification. Springer Nature Switzerland: 3-17.
- [17] LIEW D, CADAR C, DONALDSON A F, et al. Just Fuzz It: Solving Floating-Point Constraints using Coverage-Guided Fuzzing[EB/OL]. Association for Computing Machinery. 2019. <https://doi.org/10.1145/3338906.3338921>.
- [18] HE S, BARANOWSKI M, RAKAMARIĆ Z. Stochastic Local Search for Solving Floating-Point Constraints[C] // Numerical Software Verification. Springer International Publishing: 76-84.
- [19] BIERE A, HEULE M, van MAAREN H, et al. Conflict-Driven Clause Learning SAT Solvers[J]. Handbook of Satisfiability, Frontiers in Artificial Intelligence and Applications, 2009: 131-153.
- [20] MOURA L M D. Lemmas on Demand for Satisfiability Solvers[C] // .
- [21] BARRETT C, FONTAINE P, TINELLI C. The Satisfiability Modulo Theories Library (SMT-LIB)[EB/OL]. 2016. www.SMT-LIB.org.

- [22] WEBER T, CONCHON S, DÉHARBE D, et al. The SMT Competition 2015 – 2018[J]. Journal on Satisfiability, Boolean Modeling and Computation, 2019, 11: 221-259. DOI: 10.3233/SAT190123.
- [23] De MOURA L, BJØRNER N. Satisfiability Modulo Theories: An Appetizer[C] // Formal Methods: Foundations and Applications. Springer Berlin Heidelberg: 23-36.
- [24] NELSON G, OPPEN D C. Simplification by Cooperating Decision Procedures[J]. ACM Trans. Program. Lang. Syst., 1979, 1(2): 245-257. DOI: 10.1145/357073.357079.
- [25] BOZZANO M, BRUTTOMESSO R, CIMATTI A, et al. Efficient Satisfiability Modulo Theories via Delayed Theory Combination[C] // Computer Aided Verification. Springer Berlin Heidelberg: 335-349.
- [26] NELSON G, OPPEN D C. Fast Decision Procedures Based on Congruence Closure[J]. J. ACM, 1980, 27(2): 356-364. DOI: 10.1145/322186.322198.
- [27] IEEE Standard for Floating-Point Arithmetic[J]. IEEE Std 754-2008, 2008: 1-70. DOI: 10.1109/IEEESTD.2008.4610935.
- [28] VIRTANEN P, GOMMERS R, OLIPHANT T E, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python[J]. Nature Methods, 2020, 17(3): 261-272. DOI: 10.1038/s41592-019-0686-2.
- [29] Sequential Quadratic Programming[M] // NOCEDAL J, WRIGHT S J. Numerical Optimization. New York, NY: Springer New York, 1999: 526-573. DOI: 10.1007/0-387-22742-3_18.
- [30] 丛明煜王丽萍. 现代启发式算法理论研究[J]. 高技术通讯, 2003(05): 105-110.
- [31] ANDRIEU C, de FREITAS N, DOUCET A, et al. An Introduction to MCMC for Machine Learning[J]. Machine Learning, 2003, 50(1): 5-43. DOI: 10.1023/A:1020281327116.

- [32] WALES D J, DOYE J P K. Global Optimization by Basin-Hopping and the Lowest Energy Structures of Lennard-Jones Clusters Containing up to 110 Atoms[J]. The Journal of Physical Chemistry A, 1997, 101(28): 5111-5116. DOI: 10.1021/jp970984n.
- [33] JOHNSON S G. The NLOpt nonlinear-optimization package[EB/OL]. 2007. <https://github.com/stevengj/nlopt>.
- [34] FU Z, SU Z. Achieving High Coverage for Floating-Point Code via Unconstrained Programming[EB/OL]. Association for Computing Machinery. 2017. <https://doi.org/10.1145/3062341.3062383>.

致 谢

感谢王豫老师和陈谦学姐的悉心指导和帮助，感谢南京大学提供的学习环境和成长机会，感谢父母一直以来的关心爱护。

